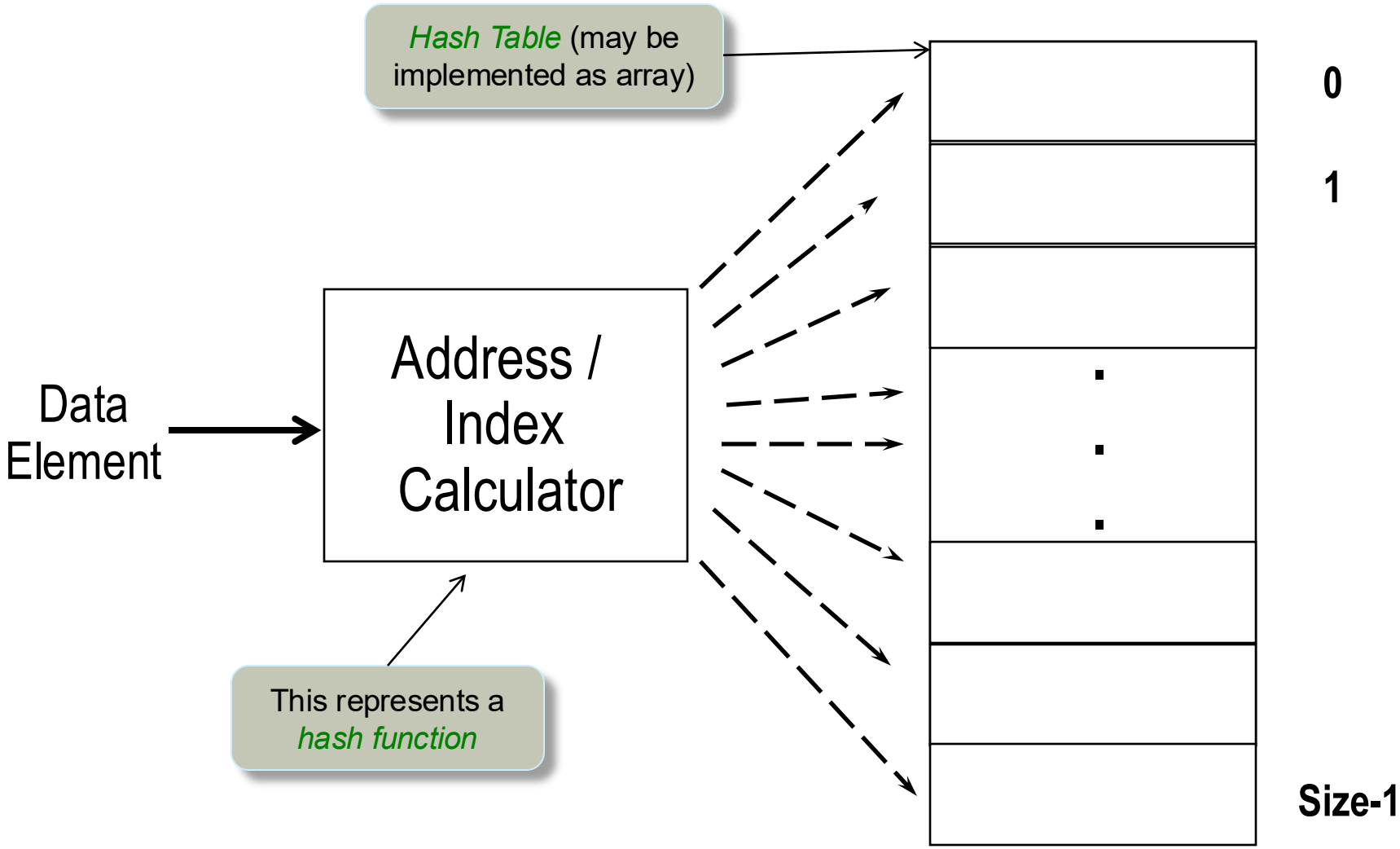
CS3021 Introduction to Data Structures and Intermediate Programming

Hash Tables & Hash Functions

Hashing

- Sorting puts the elements of a collection into some order (we'll see algorithms for this next).
- *Hashing* does the opposite by putting elements into a *random* order.
- Why do we want to do this?
  - As we saw in searching, the best performance we can expect is $O(\log n)$.
  - If we could use some kind of *formula* to calculate the exact location of the element, we could access it directly, i.e., in $O(1)$ time.

General Idea of Hashing



Address Computation

- The scheme for computing address locations for data elements in a Hash Table is called a *hash function*.
  - The same hash function is applied to *locate* any value that is already stored in the table.
- The amount of time required to store, retrieve, delete data elements in a Hash Table is very fast – essentially constant time, $O(1)$.

Hash Functions

- A *hash function* computes where the elements are stored.
  - Returns the *index* in the hash table.
- The choice of a *hash function* depends on the data being stored and accessed:
  - An analysis of the distribution of data is required to find an *optimal hashing function* for a particular data set.
  - What works efficiently for one data set may not work efficiently for another.

Types of Hash Functions (1)

- Digit Selection:
  - An exact one-to-one mapping with a hash table index is usually not practical for large key values.
  - Example:
    - Hash Index = SSN.
    - We would need a hash table with a 9-digit number of elements, e.g., “512-34-5678” hashes to 512345678.
    - But perhaps we only need to store ~10^3 unique SSNs, and not all ~10^9 possible values.
    - We could select a portion of the key (SSN), and use that as the index, e.g., *hash index* = last four digits of SSN.

Types of Hash Functions (2)

- Multiplication or Division:
  - Multiply the key by some value to produce a larger index, which helps spread indices out, e.g.,

    index = value * key
    - In this scheme, we often modulo the key in multiplying:

      index = value * (key * x mod 1), where $0 < x < 1$
  - Divide the key by the table size, e.g.,

    index = key mod *TableSize*
    - This ensures the data maps to a valid index in the table of some finite size.
- A hash table size that is a prime allows better distribution of data, avoids clustering.

Types of Hash Functions (3)

- *Folding* is technique of adding the digit values of the key to produce an index.
  - Suppose the key = 12345
  - Then index is 1 + 2 + 3 + 4 + 5 = 15
  - The digits of the key can also be multiplied by different constants to produce the index:
  - E.g., for key 12345:

    index = 1*c1 + 2*c2 + 3*c3 + 4*c4 + 5*c5

    = 1*1 + 2*2 + 3*3 + 4*4 + 5*5 = 55

Hash Collisions (1)

- When a *hash function* produces the same result for different input values, a *hash collision* occurs.
- A *hash collision* creates uncertainty about the value stored in a given hash table location.
  - Suppose we have two keys (4567, 7597) to store in a hash table of size 101, so we decide to use a modulo type *hash function*, e.g.,

    $$\text{index}(n) = n \bmod 101$$

  - Such a function would produce the same storage location for the two different data values.

Hash Collisions (2)

Table Size = 101
index(n) = n mod 101

index(**7597**) = 22

index(**4567**) = 22

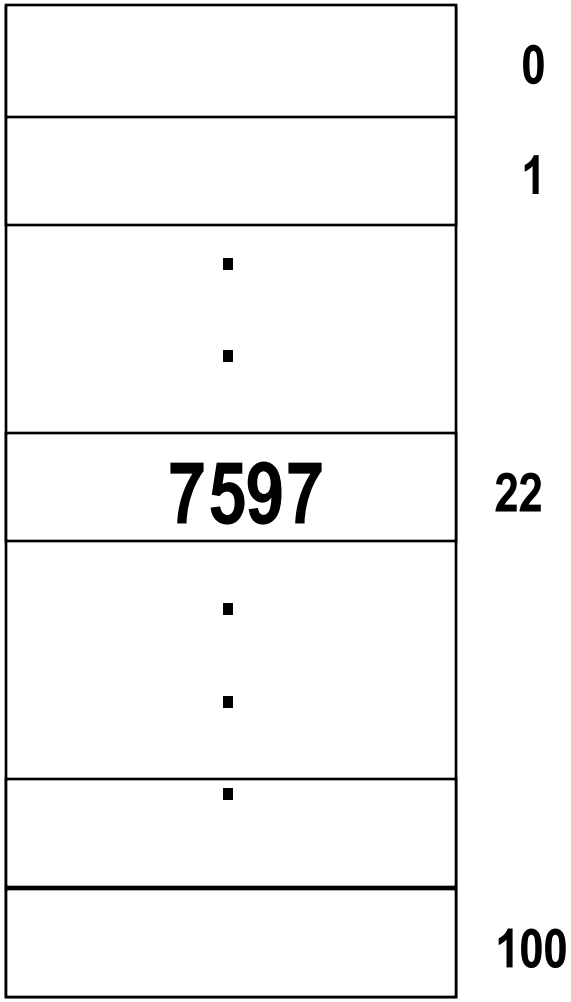


Table index 22 is already occupied, causing a *hash collision* between keys **7597** and **4567**.

Collision Resolution Schemes

- *Open Addressing*
  - Try to find an open table position after a collision.
  - Involves *probing*, or looking, for some other empty/open location in which to place the data.
  - The insertion *probe sequence* must be *reproducible* so retrieve/delete operations can always find item.
  - Various methods exist to probe for an empty/open location.
- A *probe sequence* is the sequence of locations that are examined when probing.

Linear Probing

- A constant, *c*, is added to the index until an open position is found, *c* = 1, 2, 3, ….
- Primary function:

  index = key mod *TableSize*
- Collision resolution:

  $index_c$ = (index + *c*) mod *TableSize*
- Can result in *primary clustering*, particularly with a weak *hash function*.
- Causes long probe searches and decreases overall efficiency of hashing.

Linear Probing

Table Size = 101

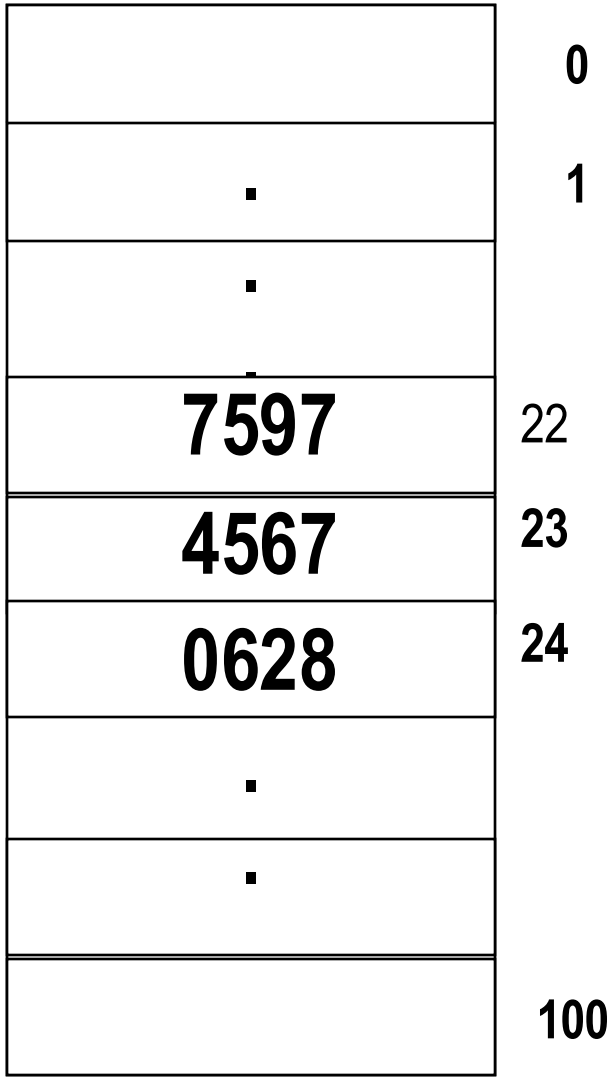
index(n) = n mod 101

c = 1, 2, 3, …

index(**7597**) = 22

index(**4567**) = 22

index(**0628**) = 22



Quadratic Probing

- A constant quadratic term, c^2, is added to the index until an open spot is found, c = 1, 2,….
- Primary index:

  index = key mod *TableSize*
- Collision resolution:

  $\text{index}_c = (\text{index} + c^2) \bmod \textit{TableSize}$
- Can result in *secondary clustering*.

Quadratic Probing

Table Size = 101

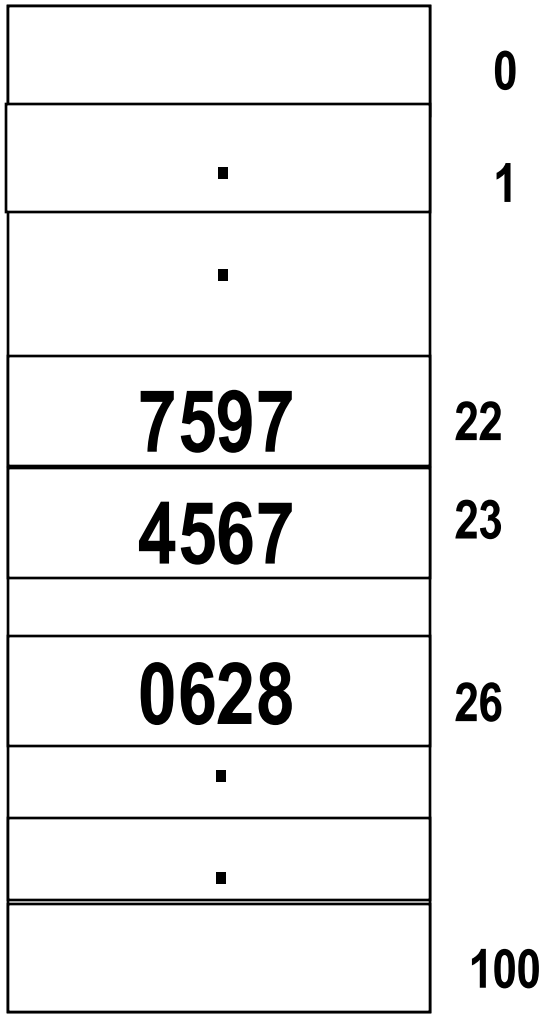
index(n) = n mod 101

c = 1, 2, 3

index(**7597**) = 22

index(**4567**) = 22 + 1^2 = 23

index(**0628**) = 22 + 2^2 = 26



Double Hashing

- Use a second hash function h_2 for probing sequence.
- The function h_2 should generate a value that is relatively prime to the table size to reduce *secondary clustering*, e.g.,

$$\text{index}_c = (\text{index} + c * h^2(\text{key}))\ \text{mod}\ \mathit{TableSize}$$

Double Hashing

Table Size = 101

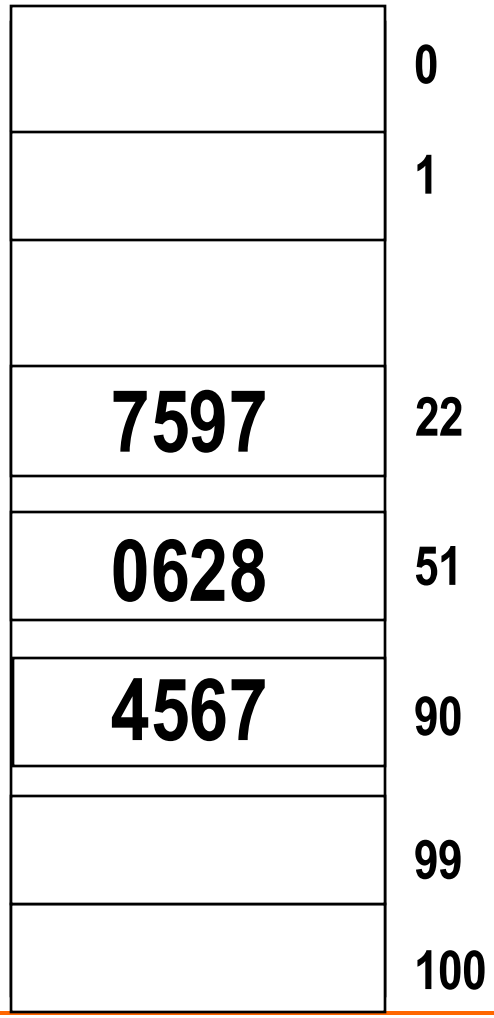
$h_1(n) = n \bmod 101$

$h_2(n) = (n \bmod 100) + 1$

index(**7597**) = 22

index(**4567**) = 22 + 68 = 90

index(**0628**) = 22 + 29 = 51



Retrieving Elements (1)

- The above *open addressing* schemes are used to *locate* elements as well as *insert* items into hash table slots.
  - But *deletion* of elements introduces a complication.
  - An open slot that is created by a previous deletion along a probe sequence could cause subsequent retrievals to fail.

Retrieving Elements (2)

Assume Linear Probing used to search for key **91**.
If search starts at index 2, linear probing will stop at index 5, since this slot is empty.

Search returns "not found" result, although **91** *is* in hash table.

. . .	0
. . .	1
58	2
105	3
24	4
<value deleted>	5
91	6
. . .	7
. . .	8
. . .	9

Retrieving Elements (3)

- Need a mechanism to signal that an open slot previously held some key value.
- We can add an add'l field to each table slot to signal its availability status:
  - AVAILABLE - previously contained data; continue searching
  - OCCUPIED - contains data; compare with search element
  - EMPTY - never had any data; element not in table

Retrieving Elements (4)

Assume Linear Probing used to search for key **91**.
When search starts at index 2, now linear probing will pass over index 5, to find **91**.

Generally, linear probing will continue until an EMPTY is discovered; "not found" result correctly returned.

. . .	0
. . .	1
58	2
105	3
24	4
AVAILABLE	5
91	6
EMPTY	7
. . .	8
. . .	9

Alternate Collision Resolution Technique

- *External Chaining*
  - Each table element is the root of a dynamic data structure (e.g., a linked list).
- Advantages over *open addressing* methods:
  - Eliminates the need for a second hashing function.
  - Prevents secondary clustering.
  - No need for availability status flag to indicate if there was an element ever in the slot.
  - Hash Table size becomes dynamic/flexible.

Questions?